

Software Requirements Specification (SRS)

Table of Contents

1. Introduction
2. Overall description
3. External interface requirements
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes and acceptance tests
7. System models and diagrams
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document specifies the software requirements for PassPulse version 2.1, a web platform for event discovery, multi-tier ticket booking, signed QR check-in and live attendance tracking. It defines functional and non-functional requirements, external interfaces and the acceptance criteria used to verify each requirement. It is the baseline for the Software Architecture and Design Specification (SAD) and the Software Test Plan (STP).

1.2 Scope

In scope: user registration and role-based access for four roles; event creation, approval and lifecycle; multi-tier ticket quotas; booking with a 10-minute inventory hold; HMAC-signed QR passes; camera-based gate check-in with duplicate rejection; a FIFO waitlist with a 15-minute claim window; a live organizer dashboard; CSV/PDF reports; and admin moderation.

Out of scope: real payment gateway integration (checkout is a mock confirmation step), native Android/iOS apps (the web client is mobile responsive), RFID or turnstile hardware, live streaming, facial recognition and ERP/accounting integration.

1.3 Audience

Course instructors and evaluators, the development team, peer reviewers from other teams, and team members acting as testers during the validation phase.

1.4 Definitions

Term	Meaning
RBAC	Role-Based Access Control. Access to routes and actions decided by the user's role.
JWT	JSON Web Token (RFC 7519). Signed bearer token used for stateless authentication.
TTL	Time-To-Live. Automatic key expiry in Redis, used for the 600-second ticket hold.
HMAC-SHA256	Keyed hash used to sign QR pass payloads so that edited passes are detected.
FIFO	First-In, First-Out ordering of the waitlist.
Hold	A temporary reservation of ticket inventory for one user during checkout.
Pass	The signed QR ticket issued for one seat of a confirmed booking.
Scan velocity	Number of successful check-ins per minute at the gates of an event.
SPA, API, SSE, TLS, CI	Single Page Application, Application Programming Interface, Server-Sent Events, Transport Layer Security, Continuous Integration.

2. Overall description

2.1 Product perspective

PassPulse is a new, self-contained web application. It replaces the mix of Google Forms, static PDF tickets and spreadsheet roll-calls that student clubs use today. The client is a React single-page application. The server is a Node.js and Express REST API. PostgreSQL stores all persistent data and Redis holds short-lived state: ticket holds and waitlist queues. Email goes out through an SMTP server via Nodemailer. The whole stack runs in Docker containers.

2.2 Major product functions

- Register, log in and enforce role permissions (Attendee, Organizer, Gate Staff, Admin)
- Search and filter published events by keyword, category, date, venue and price
- Create events, configure ticket tiers and move events through their lifecycle states
- Hold tickets for 10 minutes and confirm bookings without overselling
- Issue HMAC-signed, time-limited QR passes for confirmed tickets
- Scan passes at the gate and reject invalid, expired or duplicate passes
- Run a FIFO waitlist and offer released seats with a 15-minute claim window
- Show live check-in telemetry and export attendance and revenue reports
- Let admins approve or flag events, deactivate accounts and review audit logs

2.3 User roles and characteristics

Role	Characteristics	Main needs
Attendee	Students and the general public. Non-technical, mostly on phones. Arrive in bursts when a popular event opens for booking.	Fast search, a clear hold timer, a pass that works offline once loaded.
Organizer	Club leads and event managers. Comfortable with forms, want numbers.	Simple event wizard, tier setup, live attendance and exports.
Gate Staff	Volunteers at the entrance, often with little training, working in a queue.	One-screen scanner with an unambiguous accept/reject result in under 200 ms.
Admin	Platform administrators with elevated rights.	Moderation queue, account controls, audit log.

2.4 Operating environment

- Server: Docker containers on Ubuntu 22.04 LTS, Node.js 20.x, Express 4
- Data: PostgreSQL 16 and Redis 7 (both run from `docker-compose.yml`)
- Client: current Chrome, Firefox, Edge and Safari on desktop and mobile; camera access through the HTML5 MediaDevices API on gate devices
- Network: gate devices need at least 1 Mbps; all server clocks synchronised by NTP

2.5 Constraints

- Inventory decrements must run inside a PostgreSQL transaction guarded by a Redis hold, so the platform never sells more seats than a tier holds.
- All protected API calls use stateless JWT bearer tokens; no server-side sessions.
- Gate validation, including the signature check and the check-in write, must finish in under 200 ms.
- Code must pass ESLint, keep Jest statement and branch coverage at or above 80% and show no critical issues in SonarQube before merge to `main`.
- Secrets (`JWT_SECRET`, `QR_HMAC_SECRET`, SMTP credentials) come from environment variables and are never committed.
- Payment is simulated; no card data is collected or stored.

3. External interface requirements

3.1 User interfaces

- **Discovery and catalog:** grid of event cards with a search bar and filters for category, date range, venue and price.
- **Event wizard:** multi-step form for title, venue, date and time, description, banner image and ticket tiers.
- **Checkout and hold:** order summary with a visible 10:00 countdown and a confirm button.
- **QR pass:** mobile view with attendee name, event, tier and the QR code at a size readable from 30 cm.
- **Gate scanner:** camera viewfinder; full-screen green result for a valid pass and red with the reason (duplicate, wrong event, expired, invalid) otherwise.
- **Organizer dashboard:** check-in progress bar, scan velocity chart, export buttons.
- **Accessibility:** text contrast at least 4.5:1, all actions reachable by keyboard, result of a scan shown by text and colour, never colour alone.

3.2 Hardware interfaces

The only hardware dependency is the device camera on gate staff phones or laptops, accessed through `navigator.mediaDevices.getUserMedia` by the `html5-qrcode` library. No dedicated scanner hardware is required.

3.3 Software interfaces

- **PostgreSQL 16** through the `pg` connection pool with parameterised queries only.
- **Redis 7** for hold keys with TTL, the per-tier availability counter and the waitlist list per tier. Keyspace expiry notifications release expired holds.
- **SMTP** through Nodemailer for booking receipts and waitlist claim invitations.
- **qrcode** npm package to render the signed payload as a PNG data URL.

3.4 Communications

- REST over HTTPS with JSON bodies. TLS 1.3 is terminated at the reverse proxy in deployment; plain HTTP is allowed only on localhost during development.
- Authorization header: `Bearer <JWT>`, token lifetime 1 day (`JWT_EXPIRES_IN=1d`).
- Live dashboard updates use Server-Sent Events from `/api/telemetry/:eventId/stream`.
- Error responses follow one shape: `{ status: "error", code, message }`.

4. System features (detailed)

Each requirement has acceptance criteria and a reference to the test case in the STP. IDs follow `FR-###` and are unchanged from SRS v2.0. Priority is High, Medium or Low.

4.1 Authentication and RBAC

Description: registration, login and role checks on every protected route.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-001	The system shall let a user register with a unique email, password, full name and role.	Functional	High	All users	AC: valid input returns 201 and a user row; a repeated email returns 409 EMAIL_EXISTS. Test: TC-AUTH-01, TC-AUTH-02	Admin role cannot be self-selected; assigned by an existing admin.
FR-002	The system shall hash passwords with bcrypt (cost ≥ 10) before storing them.	Functional	High	Security	AC: stored value starts with \$2 and cost field ≥ 10 ; plaintext never in DB or logs. Test: TC-AUTH-03	bcryptjs
FR-003	The system shall authenticate requests with JWT bearer tokens and return 403 when the role is not allowed on the route.	Functional	High	Security	AC: missing or bad token gives 401; attendee token on an organizer route gives 403. Test: TC-AUTH-04, TC-AUTH-05	Needed by every module below.

4.2 Event discovery and lifecycle

Description: attendees find events; organizers create and manage them through defined states.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-004	The system shall let users filter published events by keyword, category, date range, venue and ticket price.	Functional	High	Attendee	AC: each filter alone and combined returns only matching PUBLISHED or LIVE events; an empty result shows "No events found". Test: TC-EVT-01, TC-EVT-02	Indexed columns: category, event_date.
FR-005	The system shall let organizers create, edit, schedule and cancel their own events with title, venue, capacity and date/time.	Functional	High	Organizer	AC: event saved in DRAFT; an organizer cannot edit another organizer's event (403). Test: TC-EVT-03	BR-006 limits edits after bookings exist.
FR-006	The system shall keep event state as DRAFT, PENDING_APPROVAL, PUBLISHED, LIVE, COMPLETED or CANCELLED and allow only the transitions in SAD Fig. B-3.	Functional	Medium	Organizer, Admin	AC: an illegal transition (e.g. DRAFT to LIVE) returns 409 INVALID_TRANSITION. Test: TC-EVT-04	BR-005 checked on submit.

4.3 Ticket tiers and booking engine

Description: tier quotas, the 10-minute hold and the atomic booking commit.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-007	The system shall let organizers define tiers (e.g. Early Bird, General, VIP) with their own quota and price.	Functional	High	Organizer	AC: tier with quota 0 or negative price rejected with 400. Test: TC-TCK-01	
FR-008	The system shall hold the selected tickets for exactly 10 minutes during checkout using a Redis key with TTL 600 s.	Functional	High	Business	AC: hold response carries <code>expiresAt = now + 600 s (±1 s)</code> ; held seats are not offered to others. Test: TC-BKG-01	Redis
FR-009	The system shall return held seats to stock when the hold expires without confirmation.	Functional	High	Business	AC: after 600 s the tier availability is back to its earlier value. Test: TC-BKG-02 (TTL shortened to 5 s in test config)	Keyspace notifications enabled on Redis.
FR-010	The system shall commit confirmed bookings inside one PostgreSQL transaction so that sold seats never exceed the tier quota.	Functional	High	Business	AC: 200 concurrent confirms on a 1-seat tier produce exactly 1 booking. Test: TC-BKG-03, TC-PERF-03	SELECT ... FOR UPDATE on ticket_tiers.

4.4 QR pass generation

Description: one signed, expiring pass per confirmed ticket.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-011	The system shall generate a QR pass for each confirmed ticket containing ticketId, eventId, userId, issuedAt, expiresAt and an HMAC-SHA256 signature of those fields.	Functional	High	Security	AC: <code>POST /api/qr/pass</code> returns 201 with a 64-hex-character signature and a PNG data URL; missing fields or a past expiresAt return 400. Test: TC-QR-01, TC-QR-04, TC-QR-05	Implemented in <code>qr.service.js</code> .
FR-012	The system shall show attendees a pass screen with their name, event details, tier and the QR code.	Functional	High	Attendee	AC: pass screen renders for each ticket of a booking and works after the page is cached. Test: TC-UX-02	Client page not yet built.

4.5 Gate check-in

Description: gate staff validate passes and the system records entry once.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-013	The system shall let authorised gate staff scan a pass with the device camera and verify its signature, expiry and event.	Functional	High	Gate Staff	AC: edited payload returns 400 INVALID_SIGNATURE; expired pass returns 400 EXPIRED_QR; pass for another event returns 400 WRONG_EVENT. Test: TC-QR-02, TC-QR-03, TC-SCN-03, TC-SCN-04	Verification uses <code>crypto.timingSafeEqual</code> .
FR-014	The system shall record the first valid scan as CHECKED_IN with a timestamp and reject every later scan of the same ticket with DUPLICATE_ENTRY.	Functional	High	Organizer	AC: first scan returns 200 CHECKED_IN; second returns 409 DUPLICATE_ENTRY. Test: TC-SCN-01, TC-SCN-02	Store is in memory at present; moves to <code>check_in_logs</code> table (SAD 4.6).

4.6 Waitlist

Description: queueing for sold-out tiers and reallocation of released seats.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-015	The system shall let an attendee join a FIFO waitlist when a tier has zero available seats.	Functional	Medium	Attendee	AC: join allowed only when available = 0; the same user cannot join the same tier twice. Test: TC-WTL-01	Redis list per tier.
FR-016	When a booking is cancelled, the system shall email the next waitlisted user and reserve the seat for 15 minutes.	Functional	Medium	Business	AC: head of queue receives one email with a claim link; an unclaimed slot moves to the next user after 900 s. Test: TC-WTL-02, TC-WTL-03	SMTP; BR-004.

4.7 Telemetry, reports and administration

Description: live gate figures for organizers and moderation tools for admins.

Req ID	Requirement	Type	Priority	Source	Acceptance criteria / Test case ref	Comments / Dependencies
FR-017	The system shall show live scan velocity (check-ins per minute) and attendance percentage on the organizer dashboard.	Functional	Medium	Organizer	AC: a new check-in appears on the dashboard within 3 s. Test: TC-ANL-01	SSE
FR-018	The system shall export attendance and revenue summaries as CSV and PDF.	Functional	Medium	Organizer, Admin	AC: CSV row count equals number of tickets; totals match the database. Test: TC-ANL-02	
FR-019	The system shall let admins approve, flag or remove events and deactivate accounts.	Functional	High	Admin	AC: a deactivated user's next request returns 401; only ADMIN can call <code>/api/admin/*</code> . Test: TC-ADM-01, TC-ADM-02	Actions written to <code>audit_log</code> (SEC-005).

4.8 Business rules

Rule	Definition	Enforced by / Test
BR-001	A user can book at most 5 tickets per transaction for one event.	Booking Engine / TC-BKG-04
BR-002	A hold expires 600 seconds after creation if not confirmed.	Redis TTL / TC-BKG-02
BR-003	A pass can be checked in once; later scans are rejected immediately.	Check-in Service / TC-SCN-02
BR-004	A waitlist claim invitation expires after 15 minutes and passes to the next user.	Waitlist Worker / TC-WTL-03
BR-005	An event cannot be submitted for approval without at least one tier with quota > 0.	Event Service / TC-EVT-04
BR-006	Once bookings exist, the event date cannot be changed or the event deleted without sending a cancellation notice to ticket holders.	Event Service / TC-EVT-05

5. Non-functional requirements (detailed)

NFRs are measurable and tied to tests in the STP. IDs NFR-####.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
NFR-001	95% of catalog search requests shall respond within 150 ms with 500 concurrent users.	Performance	High	k6 run of 5 minutes at 500 virtual users; p95 ≤ 150 ms. Test: TC-PERF-01
NFR-002	QR verification plus the check-in write shall complete in under 200 ms per scan.	Performance	High	p99 server time < 200 ms over 1,000 sequential scans. Test: TC-PERF-02
NFR-003	The booking engine shall handle 200 concurrent hold requests per second with no oversell.	Performance / Integrity	High	200 parallel requests on a 1-seat tier give 1 success and 199 x 409. Test: TC-PERF-03
NFR-004	The system shall be available at least 99.5% of the time during an event's booking window and on the event day.	Reliability	Medium	Health check <code>/api/health</code> polled every minute; downtime log reviewed after each event. Test: TC-OPS-01
NFR-005	An attendee shall be able to go from the home page to a confirmed booking in no more than 4 screens.	Usability	Medium	Task walkthrough with 5 first-time users; all complete in ≤ 4 screens. Test: TC-UX-01
NFR-006	Server code shall keep Jest statement and branch coverage at or above 80%.	Maintainability	High	<code>npm test -- --coverage</code> in CI; build fails below threshold. Current: 89.7% statements, 90.3% branches. Test: TC-OPS-02
NFR-007	Booking, check-in and audit records shall be kept for 1 year and survive a server restart.	Data retention	Medium	Restart containers; all rows present. Test: TC-OPS-03

5.1 Security

5.1.1 Security objectives

ID	Objective	Threat addressed
SO-1	Only the genuine holder of a confirmed ticket gets through the gate, and only once. A pass that has been copied, edited or replayed is rejected before entry.	Screenshot sharing, forged passes, replay at a second gate.
SO-2	Users can act only within their role. Attendees cannot reach organizer, gate or admin functions, and organizers can change only their own events.	Privilege escalation, insecure direct object references.
SO-3	Passwords, tokens and signing secrets are never exposed in storage, logs, responses or the repository.	Credential theft, secret leakage through git or logs.

5.1.2 Security requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
SEC-001	User passwords shall be stored only as bcrypt hashes with cost $\geq$ 10.	Security	High	DB inspection shows no plaintext; cost parsed from hash $\geq$ 10. Test: TC-AUTH-03 (SO-3)
SEC-002	RBAC middleware shall block attendee tokens from organizer, gate and admin routes with 403.	Security	High	Every protected route called with each wrong role returns 403. Test: TC-SEC-01 (SO-2)
SEC-003	All input shall be validated and all SQL parameterised to prevent SQL injection and XSS.	Security	High	OWASP ZAP baseline scan reports no High alerts; injection payloads in search return 400 or empty result. Test: TC-SEC-02
SEC-004	Client to server traffic shall use TLS 1.3 outside localhost.	Security	High	<code>curl --tls-max 1.2</code> against deployment fails; HTTP redirects to HTTPS. Test: TC-SEC-03
SEC-005	Failed logins, rejected scans and admin actions shall be written to an audit table with time, user and IP.	Security	Medium	Each listed event creates exactly one audit row. Test: TC-SEC-04
SEC-006	QR passes shall carry an HMAC-SHA256 signature over a canonical payload, compared in constant time, using a secret loaded only from <code>QR_HMAC_SECRET</code> .	Security	High	Any single changed field gives <code>INVALID_SIGNATURE</code> ; server refuses to start without the secret. Test: TC-QR-02, TC-SEC-05 (SO-1, SO-3)

6. Quality attributes and acceptance tests

- **Exit criteria for acceptance:** all High priority FRs implemented and their test cases passed; no open Critical or High defect; NFR-001 to NFR-003 met in the load test; RTM shows every requirement with at least one passed test case.
- **Acceptance test suites:** Authentication, Event Lifecycle, Booking and Concurrency, QR and Gate Check-in, Waitlist, Telemetry and Reports, Administration, Performance, Security, Usability.
- **Acceptance is run by:** the team, with the course faculty observing at the final demo. Scenario data: one test event "TechSummit 2026" with Early Bird (50), General (300) and VIP (20) tiers.

7. System models and diagrams

7.1 UML use case diagrams

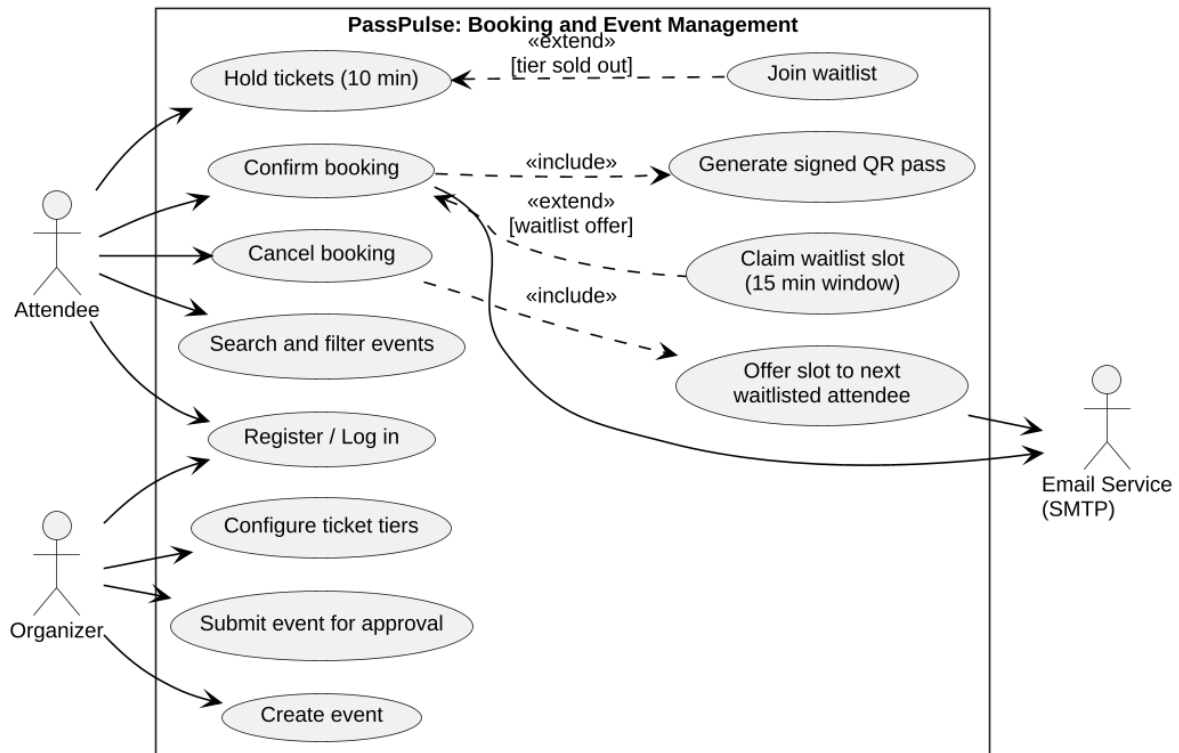


Figure A-1. Use cases for booking and event management (Attendee, Organizer)

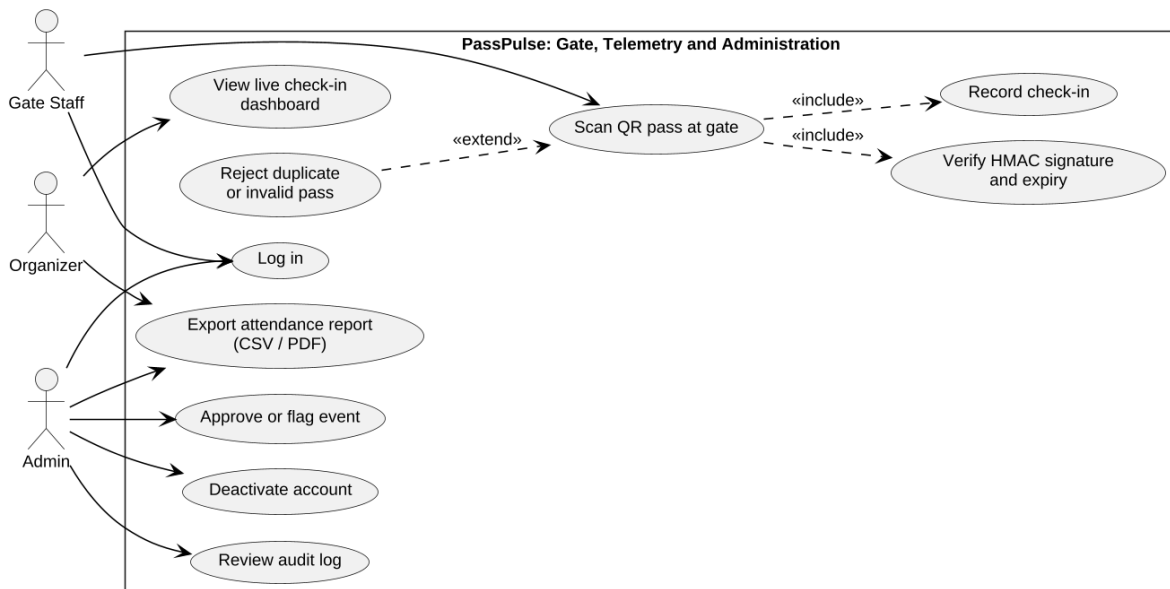


Figure A-2. Use cases for gate check-in, telemetry and administration

7.2 Use case summary

ID	Use case	Actor	Precondition	Main success outcome	Requirements
UC-01	Register / Log in	All	Valid email format	JWT issued	FR-001 to FR-003
UC-02	Search and filter events	Attendee	None	Filtered list shown	FR-004
UC-03	Create and submit event	Organizer	Logged in as organizer	Event in PENDING_APPROVAL	FR-005 to FR-007
UC-04	Hold tickets	Attendee	Seats available	Hold with 10-minute timer	FR-008, FR-009
UC-05	Confirm booking	Attendee	Active hold	Booking saved, passes issued	FR-010 to FR-012
UC-06	Join / claim waitlist	Attendee	Tier sold out	Queued; later offered seat	FR-015, FR-016
UC-07	Scan pass at gate	Gate Staff	Camera allowed, staff logged in	Attendee marked CHECKED_IN	FR-013, FR-014
UC-08	View dashboard / export	Organizer, Admin	Event PUBLISHED or later	Live figures; CSV/PDF file	FR-017, FR-018
UC-09	Moderate	Admin	Logged in as admin	Event approved or flagged	FR-019

UC-07 detailed flow (Scan pass at gate). Main flow: (1) Staff opens the scanner for the event. (2) Staff points the camera at the attendee's pass. (3) Client decodes the QR text and sends it with the event ID. (4) System checks the signature. (5) System checks expiry. (6) System checks the pass belongs to this event. (7) System checks the ticket has not been used. (8) System records the check-in and shows a green CHECKED_IN screen. Alternate flows: 4a signature mismatch: show red INVALID, log the attempt, return to step 2. 5a pass expired: show red EXPIRED. 6a wrong event: show red WRONG EVENT with the event name on the pass. 7a already used: show red DUPLICATE with the first check-in time. Exception: 3a network down: show "No connection, retry" and do not admit.

8. Requirements Traceability Matrix (RTM)

Status: **N** = not yet implemented, **P** = partly implemented or in review, **A** = implemented and all linked test cases passing. Status as of commit 4801f49.

Req ID	Requirement short	Section ref / Design spec	Module	Test case(s)	Status	Comments
FR-001	Register, unique email	4.1 / SAD 3.4, 4.3	Auth Service	TC-AUTH-01, TC-AUTH-02	N	Sprint 2 branch open
FR-002	bcrypt hashing	4.1 / SAD 3.9	Auth Service	TC-AUTH-03	N	bcryptjs added
FR-003	JWT + RBAC	4.1 / SAD 3.9	Auth Middleware	TC-AUTH-04, TC-AUTH-05	N	jsonwebtoken added
FR-004	Catalog filters	4.2 / SAD 4.3	Event Service	TC-EVT-01, TC-EVT-02	N	
FR-005	Create/edit events	4.2 / SAD 3.4	Event Service	TC-EVT-03	N	
FR-006	Lifecycle states	4.2 / SAD Fig. B-3	Event Service	TC-EVT-04	N	
FR-007	Ticket tiers	4.3 / SAD Fig. B-2	Event Service	TC-TCK-01	N	
FR-008	10-min hold	4.3 / SAD Fig. B-4	Booking Engine	TC-BKG-01	N	Redis container ready
FR-009	Hold expiry release	4.3 / SAD Fig. B-4	Booking Engine	TC-BKG-02	N	

Req ID	Requirement short	Section ref / Design spec	Module	Test case(s)	Status	Comments
FR-010	Atomic commit	4.3 / SAD Fig. B-4, ADR-02	Booking Engine	TC-BKG-03, TC-PERF-03	N	
FR-011	Signed QR pass	4.4 / SAD 4.3.1	QR Pass Service	TC-QR-01, TC-QR-04, TC-QR-05	A	qr.service.js, qr.routes
FR-012	Pass screen	4.4 / SAD 4.5	Client	TC-UX-02	N	
FR-013	Verify at gate	4.5 / SAD Fig. B-5	Check-in Service	TC-QR-02, TC-QR-03, TC-SCN-03, TC-SCN-04	A	Server side done; scanner UI pending
FR-014	Single check-in	4.5 / SAD Fig. B-5	Check-in Service	TC-SCN-01, TC-SCN-02	P	In-memory store; DB table pending
FR-015	Join waitlist	4.6 / SAD 3.6	Waitlist Worker	TC-WTL-01	N	
FR-016	Offer and claim	4.6 / SAD 3.6	Waitlist Worker	TC-WTL-02, TC-WTL-03	N	
FR-017	Live telemetry	4.7 / ADR-04	Telemetry Service	TC-ANL-01	N	
FR-018	CSV/PDF export	4.7	Telemetry Service	TC-ANL-02	N	
FR-019	Admin moderation	4.7 / SAD 3.4	Admin Service	TC-ADM-01, TC-ADM-02	N	
NFR-001	Search p95 ≤ 150 ms	5 / SAD 3.6	Event Service	TC-PERF-01	N	
NFR-002	Scan < 200 ms	5 / SAD Fig. B-5	Check-in Service	TC-PERF-02	P	Service in place; benchmark pending
NFR-003	No oversell at 200 rps	5 / ADR-02	Booking Engine	TC-PERF-03	N	
NFR-004	99.5% availability	5 / SAD 4.4	Platform	TC-OPS-01	P	/api/health exists
NFR-005	≤ 4 screens to book	5 / SAD 4.5	Client	TC-UX-01	N	
NFR-006	Coverage ≥ 80%	5 / STP 14	All server code	TC-OPS-02	A	89.7% / 90.3%
NFR-007	1-year retention	5 / SAD 3.6	PostgreSQL	TC-OPS-03	P	Named volumes in compose
SEC-001	bcrypt storage	5.1.2 / SAD 3.9	Auth Service	TC-AUTH-03	N	
SEC-002	RBAC 403	5.1.2 / SAD 3.9	Auth Middleware	TC-SEC-01	N	Not yet applied to /api/qr, /api/checkin
SEC-003	Input validation, SQLi/XSS	5.1.2 / SAD 3.9	All controllers	TC-SEC-02	P	Field checks in QR and check-in controllers
SEC-004	TLS 1.3	5.1.2 / SAD 3.6	Reverse proxy	TC-SEC-03	N	Deployment item
SEC-005	Audit log	5.1.2 / SAD 4.4	All services	TC-SEC-04	N	
SEC-006	HMAC pass, constant-time compare	5.1.2 / SAD 4.3.1	QR Pass Service	TC-QR-02, TC-SEC-05	A	

Software Architecture and Design Specification (SAD)

1. Introduction

1.1 Purpose

This document describes the architecture and detailed design of PassPulse. It explains how the requirements in SRS v2.1 are split across components, which architectural decisions were taken and why, how the main flows run, and what each implemented API looks like.

1.2 Scope

Covers the React client, the Express API and its services, PostgreSQL and Redis data stores, the SMTP integration and the security architecture. Hosting and the Jenkins pipeline are described only as far as they affect the design. Payment is a mock step and is not designed here.

1.3 Audience

Developers, testers, course instructors and evaluators, and reviewers from other teams.

1.4 Definitions

ADR (Architecture Decision Record), API, DTO (Data Transfer Object), HMAC, JWT, RBAC, SSE, STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege), TTL. Other terms as in SRS Section 1.4.

2. Document Overview

2.1 How to use this document

Section 3 covers the architecture: goals, component and class diagrams, the chosen pattern, technology, risks, traceability and the threat model. Section 4 covers the design: the event state machine, two sequence diagrams, API definitions, error handling and UX. Readers checking a single requirement should start from the traceability table in Section 3.8.

2.2 Related Documents

- PassPulse Software Requirements Specification (SRS) v2.1
- PassPulse Software Test Plan (STP) v1.0
- PassPulse RTM (SRS Section 8)
- Repository README and `docker-compose.yml`, commit 4801f49

3. Architecture

3.1 Goals and Constraints

Goals

- Zero oversold seats under burst load (NFR-003, FR-010).
- Gate decision in under 200 ms, including the database write (NFR-002).
- Passes that cannot be forged or reused (SO-1).
- Each of the four modules can be built and tested by one team member without blocking the others.

Constraints

- Small development team, one semester, student hosting budget: one VM or a free-tier container host.
- Stack fixed in the synopsis: React, Node.js/Express, PostgreSQL, Redis, Docker, Jenkins.
- No real payments; no native mobile apps.

3.2 Stakeholders and Concerns

Stakeholder	Concerns	Addressed in
Attendees	Fair access to seats during a rush; a pass that works at the gate	ADR-02, Fig. B-4, Fig. B-5
Organizers	No overbooking of the hall; live headcount; exports	ADR-02, ADR-04, 4.3
Gate staff	Fast, clear accept/reject; works on a cheap phone	Fig. B-5, 4.5
Admins	Moderation, audit trail	3.4, 3.9, 4.4
Developers	Clear module boundaries, testability, one command to run locally	3.5, 3.6
Evaluators	Traceability from requirement to code and test	3.8, SRS 8

3.3 Component (UML) Diagram

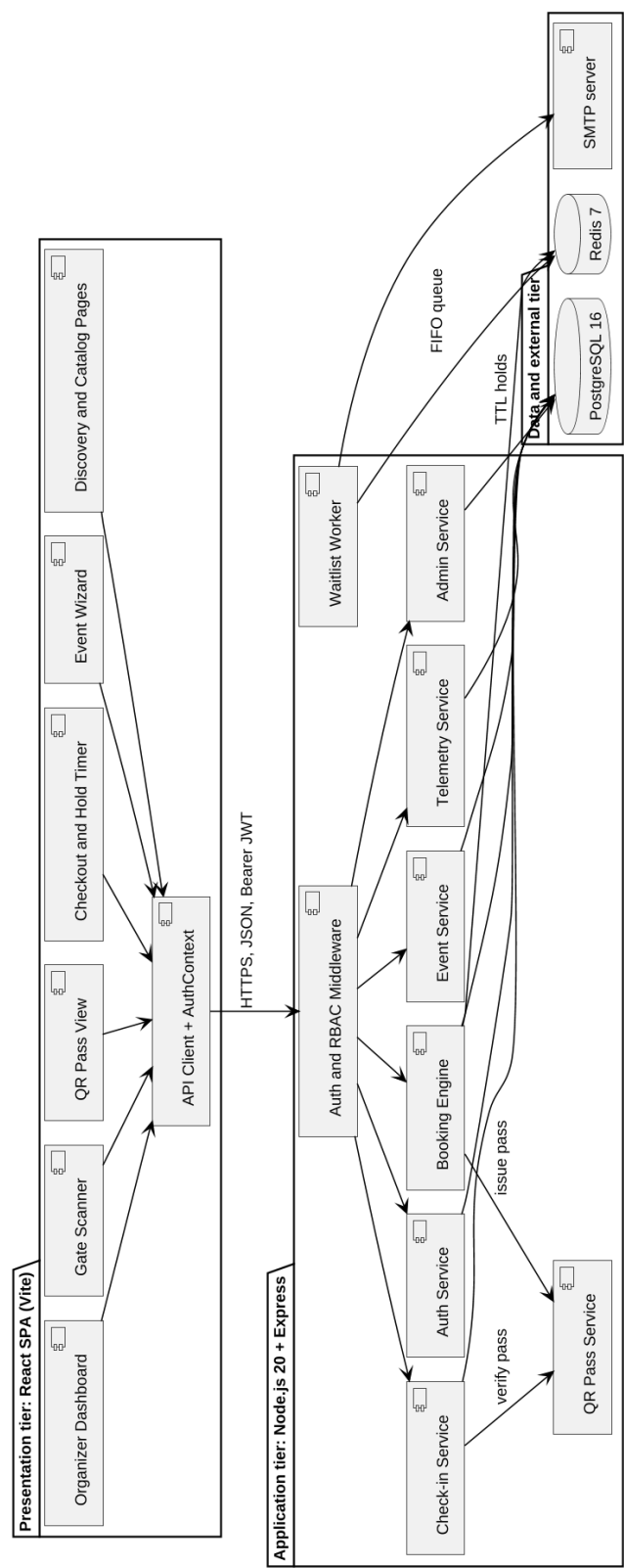


Figure B-1. Component diagram (rotated). Every client request passes through the Auth and RBAC middleware before reaching a service.

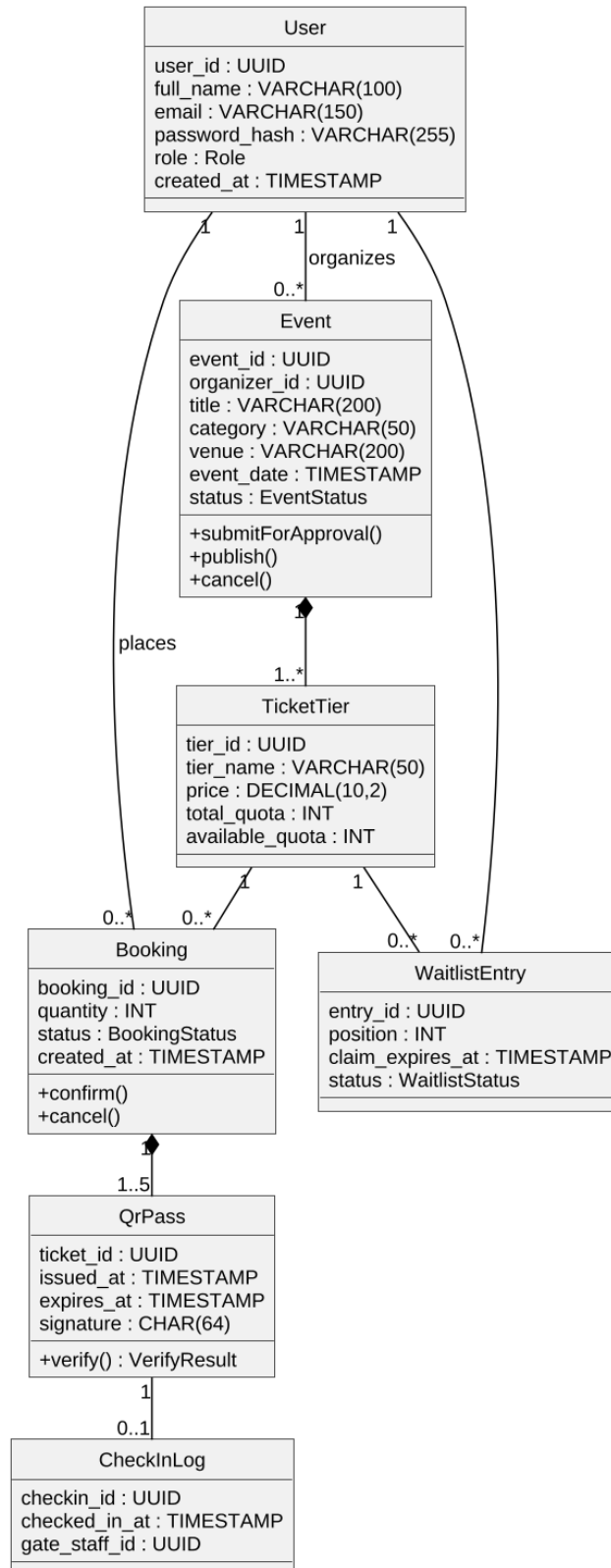


Figure B-2. Class diagram of the domain model (maps one-to-one to PostgreSQL tables)

3.4 Component Descriptions

Component	Responsibility	Status
React SPA	All screens; holds the JWT in memory through AuthContext and calls the API through one Axios client that adds the Authorization header.	Scaffold
Auth and RBAC Middleware	Verifies the JWT, loads the user's role and checks it against a per-route allow list. Rejects with 401 or 403.	Planned

Component	Responsibility	Status
Auth Service	Registration, login, bcrypt hashing, token issue.	Planned
Event Service	Event CRUD, tier setup, lifecycle transitions, catalog search.	Planned
Booking Engine	Creates Redis holds, confirms bookings in one PostgreSQL transaction, calls the QR Pass Service, handles cancellation.	Planned
Waitlist Worker	Background process that listens for released seats, pops the next user from the tier queue, sends the claim email and enforces the 15-minute window.	Planned
QR Pass Service	Builds the canonical payload, signs it with HMAC-SHA256, renders the QR image and verifies scanned passes.	Implemented
Check-in Service	Calls the QR Pass Service to verify, checks event match and prior use, records the check-in.	Implemented (in-memory store)
Telemetry Service	Aggregates check-ins per minute and pushes updates over SSE; builds CSV/PDF reports.	Planned
Admin Service	Approve or flag events, deactivate accounts, query audit log.	Planned
PostgreSQL 16	System of record for users, events, tiers, bookings, passes, check-ins, audit log.	Container ready
Redis 7	Hold keys with TTL, per-tier availability counter, waitlist lists.	Container ready

3.5 Chosen Architecture Pattern and Rationale

Layered client-server architecture, deployed as one API process (modular monolith). The client is the presentation layer. On the server each module has three layers: routes and controllers (HTTP, validation, status codes), services (business rules), and data access (SQL and Redis calls). A layer only calls the one below it. This is already the structure in the repository: `routes/`, `controllers/`, `services/`, `models/`.

Reasons: each module can be owned as one vertical slice and can unit test the service layer without HTTP (the existing `qr.service.test.js` and `checkin.service.test.js` do this). One process keeps transactions simple, which matters for the booking commit. Deployment is one `docker-compose` up.

Rejected alternatives. Microservices: a small team cannot run and monitor nine services, and a booking would need a distributed transaction or saga between booking, inventory and pass services. Serverless functions: cold starts would break the 200 ms gate target, and Redis keyspace listeners need a long-running process.

Architecture Decision Records

ADR	Decision	Context and reason	Consequence
ADR-01	Layered modular monolith	See above.	Must keep module boundaries by code review, since nothing enforces them at runtime.
ADR-02	Redis TTL hold in front of a PostgreSQL transaction for booking	A database row lock alone held for 10 minutes would block the tier. Redis gives a cheap expiring hold; the final commit still uses <code>SELECT ... FOR UPDATE</code> so PostgreSQL is the source of truth even if Redis is wrong.	Two stores can disagree briefly; a nightly job reconciles <code>avail:{tierId}</code> with the table.
ADR-03	HMAC-SHA256 signed JSON payload in the QR, not a JWT and not a bare ticket ID	A bare ID can be guessed and copied. A JWT works but is longer, making a denser QR that scans slower on low-end phones. HMAC over five fields gives a short payload and tamper detection with one secret.	Secret rotation invalidates all issued passes; rotation only between events.
ADR-04	Server-Sent Events for live dashboard	Updates go one way, server to organizer. SSE runs over plain HTTP and reconnects automatically; WebSockets add a second protocol for no gain here.	Max about six SSE connections per browser per domain over HTTP/1.1; fine for one dashboard tab.

3.6 Technology Stack and Data Stores

Layer	Technology (version from package.json / compose)
Client	React 19, Vite 8, Tailwind CSS, html5-qrcode, Axios; linted with oxlint
API	Node.js 20, Express 4.19, jsonwebtoken 9, bcryptjs 2.4, qrcode 1.5, pg 8.12, dotenv, cors
Data	PostgreSQL 16-alpine, Redis 7-alpine, both with named Docker volumes
Testing	Jest 29, Supertest 7, k6, OWASP ZAP
CI / quality	GitHub Actions and Jenkins, ESLint, SonarQube

Redis key design

Key	Type	TTL	Purpose
<code>hold:{holdId}</code>	Hash	600 s	userId, tierId, qty for one checkout (FR-008)
<code>avail:{tierId}</code>	String (int)	none	Fast availability counter; DECRBY on hold, INCRBY on expiry
<code>wl:{tierId}</code>	List	none	FIFO waitlist of userIds (RPUSH / LPOP)
<code>claim:{tierId}:{userId}</code>	String	900 s	Open waitlist offer (FR-016, BR-004)

PostgreSQL tables: users, events, ticket_tiers, bookings, tickets (one row per QR pass), check_in_logs (unique index on ticket_id, which enforces BR-003 at the database level), waitlist_entries, audit_log. Foreign keys as in Figure B-2.

3.7 Risks and Mitigations

Risk	Impact	Mitigation
Redis restarts and loses hold keys mid-checkout	Medium	Final commit re-checks quota in PostgreSQL under row lock, so the worst case is a failed checkout, never an oversell. Enable AOF persistence.
Gate network is slow or drops at the venue	High	Small request body (QR text only), 3 s client timeout with a retry button; never admit on timeout.
QR_HMAC_SECRET leaks	High	Secret only in env and the CI secret store; .env in .gitignore; rotate between events.
Check-in store is in memory in the current build	High	Move to check_in_logs table with unique ticket_id before Sprint 5 ends (Open Issue OI-1).
One module falls behind and blocks integration	Medium	Services exchange plain DTOs; each module has stubs so others can integrate early.

3.8 Traceability to Requirements

Requirement	Component	Design reference
FR-001 to FR-003, SEC-001, SEC-002	Auth Service, Auth Middleware	3.4, 3.9, 4.3.4
FR-004 to FR-007, BR-005, BR-006	Event Service	3.4, Fig. B-2, Fig. B-3
FR-008 to FR-010, NFR-003, BR-001, BR-002	Booking Engine, Redis, PostgreSQL	ADR-02, Fig. B-4, 4.3.3
FR-011, SEC-006	QR Pass Service	ADR-03, 4.3.1
FR-013, FR-014, NFR-002, BR-003	Check-in Service	Fig. B-5, 4.3.2
FR-015, FR-016, BR-004	Waitlist Worker	3.6 (Redis keys)
FR-017, FR-018	Telemetry Service	ADR-04
FR-019, SEC-005	Admin Service, audit_log	3.4, 4.4

3.9 Security Architecture

Trust boundaries. (1) Between the browser and the API: nothing from the client is trusted, including role claims in the body. (2) Between the API and the data stores: only the API container can reach PostgreSQL and Redis on the Docker network; their ports are not published in production. (3) Between gate devices and the API: a gate device is a client like any other and must hold a STAFF token.

Threat model (STRIDE)

Threat	Example against PassPulse	Control	Req
Spoofing	Attendee forges a token claiming ORGANIZER	JWT signed with JWT_SECRET; role read from DB on each request, not from the body	FR-003, SEC-002
Tampering	Attendee edits the eventId or expiresAt in a pass	HMAC-SHA256 over canonical payload; constant-time compare	SEC-006
Repudiation	Gate staff claims a guest was never admitted	check_in_logs stores ticket, time and staff ID; audit_log for rejected scans	SEC-005
Information disclosure	Password hashes or secrets in logs or API responses	bcrypt; response DTOs exclude password_hash; secrets only in env	SEC-001, SO-3
Denial of service	Bot floods the hold endpoint to lock all seats	Max 5 seats per hold (BR-001), one active hold per user per tier, rate limit 10 holds/min per user	NFR-003, BR-001
Elevation of privilege	Organizer edits another organizer's event by changing the ID in the URL	Service checks event.organizer_id = req.user.id before any write	FR-005, SEC-002

Replay of a valid pass is handled by the single-use check (BR-003) rather than by the signature, since a screenshot of a genuine pass carries a genuine signature.

4. Design

4.1 Design Overview

Each module follows the same pattern: a route file declares the path, a controller validates the body and maps results to HTTP codes, and a service holds the logic and returns plain result objects such as `{ success, reason }`. Controllers never throw business errors to Express; they translate reason codes using a lookup table. This is how the two implemented controllers work today. Event state changes are restricted to the transitions in Figure B-3.

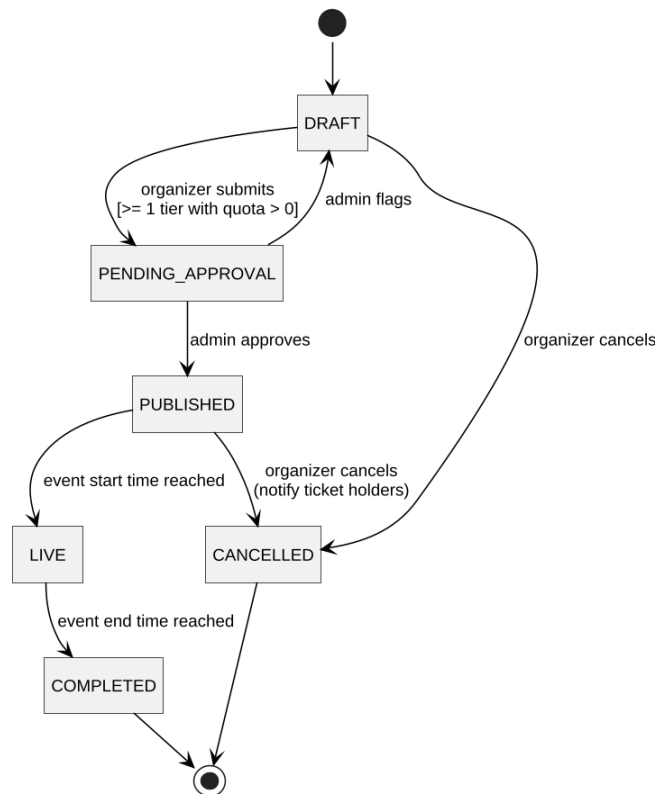


Figure B-3. Event lifecycle state machine (FR-006, BR-005)

4.2 UML Sequence Diagrams

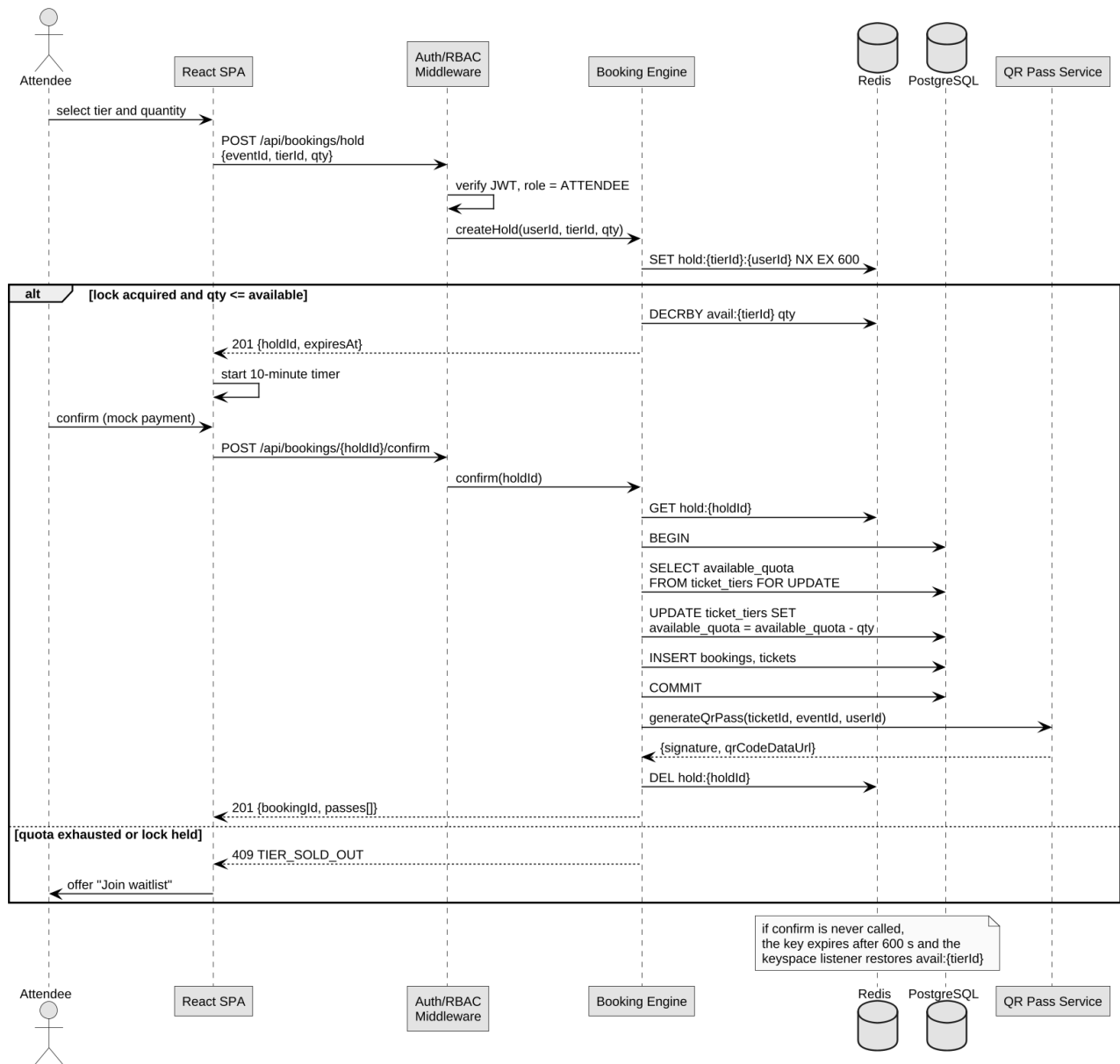


Figure B-4. Hold and confirm booking (FR-008, FR-009, FR-010, FR-011)

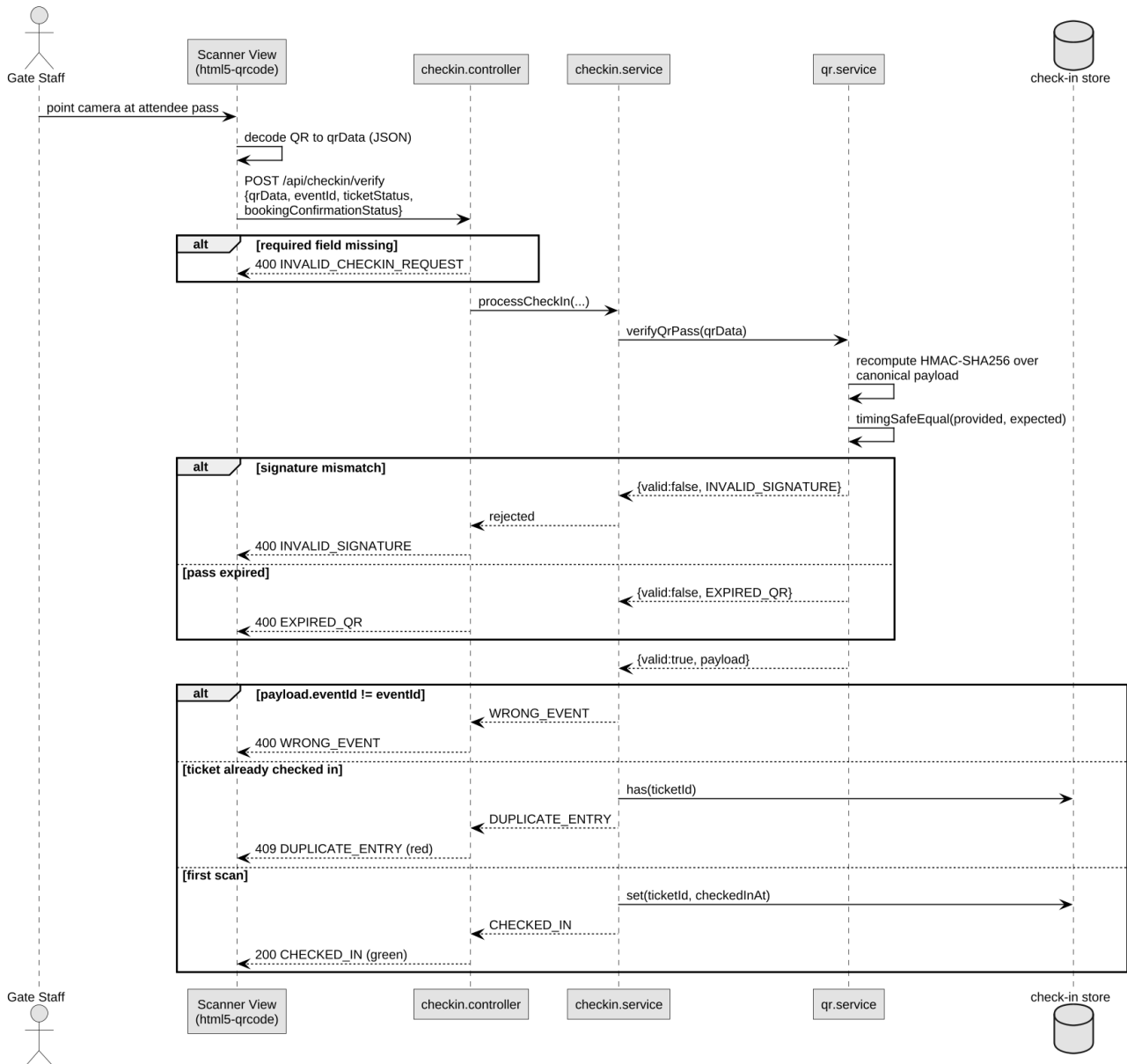


Figure B-5. Gate check-in as implemented in commit 4801f49 (FR-013, FR-014)

4.3 API Design

Base path `/api`. All bodies are JSON. Errors use `{ "status": "error", "code": "...", "message": "...", "data": {} }`. Sections 4.3.1 and 4.3.2 describe endpoints that exist in the repository; 4.3.3 and 4.3.4 are the agreed contracts for modules in progress.

4.3.1 QR Pass Service (implemented)

Endpoint	<code>/api/qr/pass</code>
Method	POST
Caller	Booking Engine after commit (internal). Will be restricted to service calls once RBAC is in place.
Request	<code>{ ticketId, eventId, userId, expiresAt }</code>
Response 201	<code>{ status: "success", data: { ticketId, eventId, userId, issuedAt, expiresAt, signature, qrData, qrCodeDataURL } }</code>
Errors	400 INVALID_QR_REQUEST (missing field); 400 INVALID_EXPIRY (expiresAt not a valid future time); 500 on internal error

Signature: `HMAC_SHA256(QR_HMAC_SECRET, JSON.stringify({ticketId, eventId, userId, issuedAt, expiresAt}))` as 64 hex characters. Key order is fixed by `serializePayload()` so that signing and verification produce the same string.

4.3.2 Check-in Service (implemented)

Endpoint	<code>/api/checkin/verify</code>
Method	POST
Caller	Gate Scanner page (role STAFF)
Request	<code>{ qrData, eventId, ticketStatus, bookingConfirmationStatus }</code>
Response 200	<code>{ status: "success", code: "CHECKED_IN", data: { ticketId, eventId, userId, checkedInAt, ... } }</code>
Errors	400 INVALID_CHECKIN_REQUEST, INVALID_QR_FORMAT, MISSING_QR_FIELDS, INVALID_SIGNATURE, EXPIRED_QR, WRONG_EVENT, MISSING_CHECKIN_FIELDS; 409 DUPLICATE_ENTRY

4.3.3 Booking Engine (contract)

Endpoint	Request / Response	Errors
POST <code>/api/bookings/hold</code>	Req <code>{ eventId, tierId, qty }</code> 201 <code>{ holdId, expiresAt }</code>	400 QTY_LIMIT (qty > 5); 409 TIER_SOLD_OUT; 409 HOLD_EXISTS
POST <code>/api/bookings/:holdId/confirm</code>	201 <code>{ bookingId, passes: [{ ticketId, qrCodeDataUrl }] }</code>	404 HOLD_EXPIRED; 409 TIER_SOLD_OUT
DELETE <code>/api/bookings/:bookingId</code>	200 <code>{ status: "CANCELLED" }</code> ; triggers waitlist offer	403 NOT_OWNER; 409 ALREADY_CHECKED_IN

4.3.4 Auth Service (contract)

Endpoint	Request / Response	Errors
POST <code>/api/auth/register</code>	Req <code>{ fullName, email, password, role }</code> 201 <code>{ userId }</code>	400 WEAK_PASSWORD; 409 EMAIL_EXISTS
POST <code>/api/auth/login</code>	Req <code>{ email, password }</code> 200 <code>{ token, role, expiresIn }</code>	401 INVALID_CREDENTIALS; 403 ACCOUNT_DISABLED

4.4 Error Handling, Logging and Monitoring

- **Error codes:** upper-case strings defined next to the service that raises them; controllers map codes to HTTP status with one lookup object (see `statusByReason` in the check-in controller). Unknown errors reach the Express error middleware, which returns 500 with a generic message and no stack trace.
- **Fail closed:** any verification error at the gate rejects entry. The scanner never shows green unless the API returns 200.
- **Logging:** JSON lines with timestamp, level, requestId, route, userId and code. Never logged: passwords, JWTs, `QR_HMAC_SECRET`, full `qrData`. The current code uses `console.error` in the error handler; this moves to a logger in Sprint 6.
- **Audit:** failed logins, rejected scans (with reason), role changes and admin actions go to the `audit_log` table (SEC-005).
- **Monitoring:** GET `/api/health` (implemented) for uptime checks; metrics tracked per event: hold success rate, confirm latency, scan latency p99, rejected scans by reason.

4.5 UX Design

- Booking path is Home, Event detail, Checkout with timer, Pass: four screens (NFR-005).

- The hold timer is always visible on checkout and turns bold under 60 seconds. On expiry the page explains the seats were released and offers to try again.
- Scanner result fills the screen with a large word (CHECKED IN, DUPLICATE, EXPIRED, WRONG EVENT, INVALID) and a colour, so staff can read it at arm's length and colour-blind staff are not dependent on colour. A short beep distinguishes accept from reject.
- The pass page shows the QR at a fixed minimum size and raises screen brightness where the browser allows.
- Forms show errors next to the field, in text, after the user leaves the field.

4.6 Open Issues and Next Steps

ID	Issue	Target
OI-1	Check-in records are kept in a JavaScript Map and are lost on restart. Replace with an INSERT into check_in_logs with a unique index on ticket_id, so the duplicate check also holds across two API instances.	Sprint 5
OI-2	The verify endpoint takes ticketStatus and bookingConfirmationStatus from the request body. These must be read from the tickets table instead of trusted from the client.	Sprint 5
OI-3	/api/qr and /api/checkin are not yet behind the Auth/RBAC middleware. Add requireRole('STAFF') to check-in and make pass generation internal only.	Sprint 3
OI-4	CORS currently allows all origins. Restrict to the client origin in production.	Sprint 6
OI-5	Decide whether passes should rotate every 30 s (time-based code) to stop screenshot sharing before the gate; current design relies on single use.	After demo

Software Test Plan (STP)

1. Introduction

Purpose: This plan defines how PassPulse v2.1 will be tested: objectives, scope, levels, test cases, environment, schedule, roles and the criteria for starting, stopping and accepting testing.

Scope: All functional requirements FR-001 to FR-019, NFR-001 to NFR-007, SEC-001 to SEC-006 and business rules BR-001 to BR-006 from SRS v2.1. Excluded items are listed in Section 4.

References: PassPulse SRS v2.1; PassPulse SAD v1.0; IEEE 829-2008 Standard for Software Test Documentation; OWASP Web Security Testing Guide v4.2; repository commit 4801f49.

Definitions: SRS, SAD, RTM, SUT (System Under Test), UAT (User Acceptance Test), TC (Test Case), p95/p99 (95th/99th percentile latency), VU (k6 virtual user).

2. Test Items

Item	Source location	Version / state
Auth and RBAC module	server/src/controllers, middleware (auth.*)	Not yet merged
Event and tier module	server/src/services (event.*)	Not yet merged
Booking engine	server/src/services (booking.*)	Not yet merged
Waitlist worker	server/src/workers	Not yet merged
QR pass service and API	server/src/services/qr.service.js, routes/qr.routes.js	main @ 4801f49
Check-in service and API	server/src/services/checkin.service.js, routes/checkin.routes.js	main @ 4801f49
Telemetry and admin	server/src/services (telemetry.*, admin.*)	Not yet merged
React client	client/src	Scaffold
Deployment configuration	docker-compose.yml, server/.env.example	main @ 4801f49

3. Features to be Tested

Req ID	Feature	Test cases
FR-001 to FR-003	Registration, password hashing, JWT and RBAC	TC-AUTH-01 to TC-AUTH-05, TC-SEC-01
FR-004 to FR-006	Catalog filters, event CRUD, lifecycle transitions	TC-EVT-01 to TC-EVT-05
FR-007	Ticket tier configuration	TC-TCK-01
FR-008 to FR-010	10-minute hold, expiry release, atomic commit	TC-BKG-01 to TC-BKG-04
FR-011, FR-012	Signed QR pass generation and pass screen	TC-QR-01 to TC-QR-06, TC-UX-02
FR-013, FR-014	Gate verification and single check-in	TC-SCN-01 to TC-SCN-05
FR-015, FR-016	Waitlist join, offer and claim window	TC-WTL-01 to TC-WTL-03
FR-017 to FR-019	Live telemetry, exports, admin moderation	TC-ANL-01, TC-ANL-02, TC-ADM-01, TC-ADM-02
NFR-001 to NFR-003	Search latency, scan latency, booking concurrency	TC-PERF-01 to TC-PERF-03
NFR-004, NFR-006, NFR-007	Availability, coverage, data retention	TC-OPS-01 to TC-OPS-03
NFR-005	Screens to complete a booking	TC-UX-01

Req ID	Feature	Test cases
SEC-001 to SEC-006	Security requirements	TC-SEC-01 to TC-SEC-05, TC-AUTH-03, TC-QR-02

4. Features Not to be Tested

- Payment processing: checkout uses a mock confirmation; no gateway exists to test.
- Internal behaviour of PostgreSQL, Redis, Nodemailer and the qrcode library; they are used as black boxes and only their integration is tested.
- Delivery to real inboxes; SMTP is pointed at a local catcher (MailHog) and only the outgoing message is checked.
- Browser compatibility beyond the four browsers in SRS 2.4; native app stores.
- Camera hardware quality. Scanner tests feed known QR images through the decoder.

5. Test Approach / Strategy

Levels

- **Unit:** Jest tests on service functions with no HTTP and no network (for example `verifyQrPass`, `processCheckIn`). Written by the module owner in the same pull request as the code.
- **Integration:** Supertest against the Express app, and service tests against real PostgreSQL and Redis containers started by docker-compose in CI.
- **System:** end-to-end flows through the React client on a staging stack: register, create event, book, scan, report.
- **Acceptance:** scenario walkthrough by the whole team with the faculty demo script, using the TechSummit 2026 data set.

Types

- Functional (black box): equivalence classes and boundary values, e.g. qty 0, 1, 5, 6 for BR-001; hold expiry at 599 s and 601 s.
- Negative: missing fields, wrong role, tampered pass, expired pass, wrong event.
- Regression: the full Jest suite runs on every push and pull request; merge blocked on failure.
- Concurrency and performance: k6 scripts for NFR-001 to NFR-003.
- Usability: task-based walkthrough with five first-time users for NFR-005.
- Static analysis: ESLint and SonarQube quality gate in CI.

Entry criteria: feature merged to `develop` with its unit tests passing; containers start from docker-compose; test data seed script runs without error.

Exit criteria: 100% of planned test cases executed; all High priority test cases passed; 0 open Critical defects and no more than 2 open Major defects with a documented workaround; statement and branch coverage $\geq 80\%$; NFR-001 to NFR-003 targets met in the final k6 run.

5.1 Security Validation

Req	What is validated	Method / tool	When
SEC-006	Edited pass fields rejected; signature compared in constant time; server will not start without QR_HMAC_SECRET	Jest (automated), code review	Every build
BR-003	Replay of a genuine pass (screenshot) rejected on second scan, also across two API instances once OI-1 is closed	Supertest, manual two-device test	Sprint 5
SEC-002	Every protected route called with each wrong role returns 403; organizer cannot edit another organizer's event by changing the ID	Postman collection with 4 role tokens	Sprint 3 onward
SEC-003	SQL injection and XSS payloads in search, event fields and login	OWASP ZAP baseline + manual payload list	Sprint 6
SEC-001	No plaintext passwords; bcrypt cost ≥ 10	DB inspection script	Sprint 2
SEC-004	TLS 1.3 only; HTTP redirects to HTTPS	curl, testssl.sh	Deployment
SEC-005	Failed logins and rejected scans written to audit_log	Supertest + DB check	Sprint 6
All	Malformed and random input to <code>/api/checkin/verify</code> and <code>/api/qr/pass</code> never returns 200 or 500	Fuzzing with a Jest property script (5,000 random bodies)	Sprint 5
All	No secrets in the repository history	gitleaks in CI	Every build

5.2 Test Case Catalogue

Priority H/M/L. Auto = automated in Jest/Supertest or k6; Man = manual. Status at time of writing: Pass, Fail, or NR (not run, feature not merged).

TC ID	Req	Description and expected result	Level	Pri	Mode	Status
TC-AUTH-01	FR-001	Register with valid data: 201, user row created with role ATTENDEE	Integ	H	Auto	NR
TC-AUTH-02	FR-001	Register with an existing email: 409 EMAIL_EXISTS	Integ	H	Auto	NR
TC-AUTH-03	FR-002, SEC-001	Stored password is a bcrypt hash with cost ≥ 10 ; plaintext not found in DB dump or logs	Integ	H	Auto	NR
TC-AUTH-04	FR-003	Protected route with no token or a token signed with another secret: 401	Integ	H	Auto	NR
TC-AUTH-05	FR-003	Attendee token on POST <code>/api/events</code> : 403	Integ	H	Auto	NR
TC-EVT-01	FR-004	Filter by category "Tech": only Tech events returned	Integ	H	Auto	NR
TC-EVT-02	FR-004	Combined filters (date range + max price + venue); DRAFT events never returned	Integ	H	Auto	NR
TC-EVT-03	FR-005	Organizer A cannot edit organizer B's event: 403	Integ	H	Auto	NR
TC-EVT-04	FR-006, BR-005	DRAFT to LIVE rejected 409; submit with no tier rejected	Unit	M	Auto	NR
TC-EVT-05	BR-006	Change date of an event with bookings: notice queued to all holders	Integ	M	Auto	NR
TC-TCK-01	FR-007	Tier with quota 0 or price -1: 400	Unit	H	Auto	NR
TC-BKG-01	FR-008	Hold 2 seats: 201, expiresAt = now + 600 s, availability reduced by 2	Integ	H	Auto	NR
TC-BKG-02	FR-009, BR-002	Let hold expire (TTL set to 5 s in test): availability restored	Integ	H	Auto	NR
TC-BKG-03	FR-010	Confirm after quota changed underneath: transaction rolls back, 409, no booking row	Integ	H	Auto	NR

TC ID	Req	Description and expected result	Level	Pri	Mode	Status
TC-BKG-04	BR-001	Boundary qty 5 accepted, qty 6 rejected 400 QTY_LIMIT	Unit	M	Auto	NR
TC-QR-01	FR-011	Generate pass for valid input: 201, 64-hex signature, PNG data URL, payload echoed	Unit + Integ	H	Auto	Pass
TC-QR-02	FR-013, SEC-006	Change eventId in a signed pass: INVALID_SIGNATURE	Unit	H	Auto	Pass
TC-QR-03	FR-013	Verify a pass whose expiresAt is in the past: EXPIRED_QR	Unit	H	Auto	Pass
TC-QR-04	FR-011	POST /api/qr/pass without userId: 400 INVALID_QR_REQUEST	Integ	H	Auto	Pass
TC-QR-05	FR-011	POST /api/qr/pass with expiresAt in the past: 400 INVALID_EXPIRY	Integ	M	Auto	Pass
TC-QR-06	SEC-003	Verify QR data with missing fields: MISSING_QR_FIELDS	Unit	M	Auto	Pass
TC-SCN-01	FR-014	First scan of a valid pass: 200 CHECKED_IN with checkedInAt	Unit + Integ	H	Auto	Pass
TC-SCN-02	FR-014, BR-003	Second scan of the same pass: 409 DUPLICATE_ENTRY	Unit + Integ	H	Auto	Pass
TC-SCN-03	FR-013	Pass for event A scanned at event B: 400 WRONG_EVENT	Unit + Integ	H	Auto	Pass
TC-SCN-04	FR-013	Pass with altered signature at API: 400 INVALID_SIGNATURE	Integ	H	Auto	Pass
TC-SCN-05	SEC-003	Check-in with missing booking status fields: MISSING_CHECKIN_FIELDS	Unit	M	Auto	Pass
TC-WTL-01	FR-015	Join waitlist when available > 0 rejected; when 0 accepted; second join by same user rejected	Integ	M	Auto	NR
TC-WTL-02	FR-016	Cancel a booking: first user in queue gets exactly one email	Integ	M	Auto	NR
TC-WTL-03	FR-016, BR-004	Offer not claimed in window (15 s in test): offer moves to second user	Integ	M	Auto	NR
TC-ANL-01	FR-017	Scan a pass: dashboard count updates within 3 s	System	M	Man	NR
TC-ANL-02	FR-018	Export CSV: row count and revenue total match DB query	Integ	M	Auto	NR
TC-ADM-01	FR-019	Admin approves event: state PUBLISHED; non-admin gets 403	Integ	H	Auto	NR
TC-ADM-02	FR-019	Deactivated user's next request: 401	Integ	H	Auto	NR
TC-PERF-01	NFR-001	k6, 500 VUs, 5 min on GET /api/events?category=Tech: p95 ≤ 150 ms	System	H	Auto	NR
TC-PERF-02	NFR-002	1,000 sequential valid scans: server p99 < 200 ms	System	H	Auto	NR
TC-PERF-03	NFR-003, FR-010	200 parallel holds on a 1-seat tier: exactly 1 x 201, 199 x 409, quota never negative	System	H	Auto	NR
TC-OPS-01	NFR-004	Health endpoint polled each minute through booking window; uptime ≥ 99.5%	System	M	Auto	NR
TC-OPS-02	NFR-006	Coverage report ≥ 80% statements and branches	Unit	H	Auto	Pass
TC-OPS-03	NFR-007	Restart all containers: bookings and check-ins still present	System	M	Man	NR
TC-UX-01	NFR-005	Five first-time users book a ticket; count screens; all ≤ 4	Accept	M	Man	NR

TC ID	Req	Description and expected result	Level	Pri	Mode	Status
TC-UX-02	FR-012	Pass screen shows name, event, tier and a scannable QR on a 5-inch phone	System	H	Man	NR
TC-SEC-01	SEC-002	Role matrix: each protected route with each wrong role returns 403	Integ	H	Auto	NR
TC-SEC-02	SEC-003	ZAP baseline: no High alerts; injection strings in search return 400 or empty list	System	H	Auto	NR
TC-SEC-03	SEC-004	TLS 1.2 handshake refused; http:// redirects to https://	System	H	Man	NR
TC-SEC-04	SEC-005	3 failed logins and 1 rejected scan produce 4 audit rows	Integ	M	Auto	NR
TC-SEC-05	SEC-006	Start server with QR_HMAC_SECRET unset: process exits with configuration error	Unit	M	Auto	NR

5.3 Sample Detailed Test Case

Field	TC-SCN-02: Reject duplicate check-in
Module	Check-in Service / API
Requirement	FR-014, BR-003
Tester	
Preconditions	Server running with QR_HMAC_SECRET set; check-in store cleared (<code>clearCheckInRecords()</code> in <code>beforeEach</code>)
Test data	ticketId T-1001, eventId E-42, userId U-7, expiresAt now + 1 hour; ticketStatus ACTIVE; bookingConfirmationStatus CONFIRMED
Steps	1. Generate a pass with <code>generateQrPass()</code> . 2. POST <code>/api/checkin/verify</code> with <code>qrData</code> and eventId E-42. 3. Repeat step 2 with the same body.
Expected	Step 2: 200, code CHECKED_IN. Step 3: 409, code DUPLICATE_ENTRY.
Actual	As expected (<code>src/routes/checkin.routes.test.js</code>)
Result	Pass

6. Test Environment

Item	Details
Hardware	Developer laptops (8 GB RAM minimum); staging VM with 2 vCPU and 4 GB RAM; two Android phones and one iPhone for scanner and pass tests
Software	Node.js 20, PostgreSQL 16-alpine and Redis 7-alpine via docker-compose, MailHog for SMTP, Chrome / Firefox / Edge / Safari
Tools	Jest 29 and Supertest 7 (unit, integration), Postman (API, role matrix), k6 (load), OWASP ZAP (DAST), SonarQube and ESLint (static), gitleaks (secrets), GitHub Projects board for defects
CI	GitHub Actions on every push and pull request: <code>install</code> , <code>lint</code> , <code>npm test -- --coverage</code> ; Jenkins nightly job adds k6 smoke test and ZAP baseline
Test data	Seed script creates 4 users per role, event "TechSummit 2026" with tiers Early Bird (50 @ 199), General (300 @ 399), VIP (20 @ 799), plus one 1-seat tier for concurrency tests. No real personal data.

7. Test Schedule

Milestone	Timeline
Unit tests for QR and check-in modules	Completed (Sprint 3)
Test case design (all modules)	End of Sprint 3
Environment setup (staging VM, MailHog, k6, ZAP)	End of Sprint 3
Integration testing per merged module	Sprints 4 and 5
System, performance and security testing	Sprint 6, first week
Regression and defect fix cycle	Sprint 6, second week
UAT with demo script	Sprint 6, final days
Test summary report	End of Sprint 6

8. Test Deliverables

- Test Plan (this document)
- Test case catalogue (Section 5.2) and Postman collection
- Jest/Supertest test files in `server/src/**/*.test.js` and k6 scripts in `server/tests/load`
- Seed data script
- CI run logs and coverage reports (HTML from Jest)
- ZAP and SonarQube reports
- Defect reports (GitHub issues labelled `bug`, with requirement ID in the title)
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead		Owens this plan, CI test stage, daily status; QR and check-in tests
Security Test Engineer		Auth, RBAC and security validation (5.1), ZAP and gitleaks
Performance Test Engineer		Booking and waitlist tests, k6 scripts, concurrency runs
Product Owner / Test Engineer		Event module tests, usability walkthrough, sign-off of acceptance
Developers (all)		Unit tests with each pull request; fix and retest assigned defects

10. Risks and Mitigation

Risk	Mitigation
Modules merged late leave little time for integration testing	Contracts in SAD 4.3 agreed early; tests written against stubs; integration starts per module as soon as it merges
Load tests on a laptop do not reflect a real server	Run k6 from a separate machine against the staging VM; record hardware with each result
Flaky tests due to Redis TTL timing	Shorten TTLs through config in test and use fake timers where possible
Staging VM unavailable	Full stack also runs locally with docker-compose
Same developer writes code and tests	Every pull request reviewed by a second member who also reviews the tests

11. Assumptions and Dependencies

- docker-compose brings up PostgreSQL 16 and Redis 7 on all team machines.
- Seed data is ready before integration testing starts.
- At least two phones with working cameras are available for scanner testing.
- Redis keyspace notifications can be enabled in the test container.
- Five students outside the team agree to take part in the usability walkthrough.

12. Suspension and Resumption Criteria

Suspend testing if:

- The build fails to start or more than 30% of planned test cases in the cycle are blocked.
- A Critical defect in auth or booking is found (for example, an oversell or an RBAC bypass); dependent tests stop until fixed.
- The test environment is unavailable for more than 4 hours.

Resume testing when: the blocking defect is fixed and its test passes, a smoke test of the affected module passes, and the environment has been stable for one hour. Blocked test cases are re-run from the start.

13. Test Case Management and Traceability

Every test case carries the requirement ID it verifies, and every requirement has at least one test case. The full mapping and implementation status is the RTM in SRS Section 8. Test case IDs follow TC-<MODULE>-NN. Automated tests include the TC ID in the Jest test name so that CI output can be matched to the RTM.

Requirement	Test cases
FR-011 (signed pass)	TC-QR-01, TC-QR-04, TC-QR-05
FR-013 (verify at gate)	TC-QR-02, TC-QR-03, TC-SCN-03, TC-SCN-04
FR-014 (single check-in)	TC-SCN-01, TC-SCN-02
FR-010 (atomic commit)	TC-BKG-03, TC-PERF-03
NFR-002 (scan < 200 ms)	TC-PERF-02
SEC-002 (RBAC)	TC-AUTH-05, TC-SEC-01

14. Test Metrics and Reporting

Metrics collected: % test cases executed; % passed and failed; defect density (defects per KLOC); defect ageing (days open by severity); requirement coverage (% of requirements with at least one passed test); code coverage (statements, branches).

Execution status at time of writing

Measure	Value
Test cases in catalogue	46
Executed / passed / failed	12 / 12 / 0
Automated Jest tests (4 suites)	15 run, 15 passed
Statement / branch / function coverage	89.71% / 90.32% / 75.00%
Requirements with at least one passed test	6 of 32 (FR-011, FR-013, FR-014, SEC-003 partly, SEC-006, NFR-006). FR-014 verified against the in-memory store only.
Open defects	0 Critical, 3 Major (OI-1, OI-2, OI-3), 1 Minor (OI-4)